EXPERIMENT - 7

AIM:

Build an Artificial Neural Network (ANN) using Keras/TensorFlow

1. Dataset Source

Dataset Name: Churn Modelling
File Used: Churn Modelling

2. Dataset Description

The dataset contains details of bank customers, used to predict the likelihood of a customer leaving the bank (Churn).

- **Type:** Numerical and Categorical
- **Domain:** Banking / CRM Analytics
- **Target:** Supervised (Binary Classification: 0 = Stayed, 1 = Exited)
- **Key Features:**

Feature	Description	Data Type
CreditScore	Credit score of the customer representing financial stability	Numerical
Geography	Country of residence (France, Spain, Germany)	Categorical
Gender	Gender of the customer	Categorical
Age	Age of the customer	Numerical
Tenure	Number of years the customer has been with the bank	Numerical
Balance	Account balance available in the bank	Numerical

NumOfProducts	Number of bank products (loans, cards, etc.) used	Numerical
HasCrCard	Whether the customer possesses a credit card (0/1)	Binary
IsActiveMember	Whether the customer is an active user of bank services	Binary
EstimatedSalary	Estimated annual salary of the customer	Numerical
Exited (Target)	Whether the customer left the bank (1) or stayed (0)	Binary

3.Mathematical Formulation of the Algorithm

A. Artificial Neural Network (ANN) An ANN consists of an input layer, hidden layers, and an output layer. Each connection has an associated weight $\boldsymbol{w}$ and bias $\boldsymbol{b}$.

Activation Functions: * ReLU (Rectified Linear Unit): Used in hidden layers to handle non-linearity.

$$\boxed{f(x) = \max(0, x)}$$

Sigmoid: Used in the output layer for binary probability.

$$\boxed{f(x) = \frac{1}{1 + e^{-x}}}$$

Optimization (Gradient Descent): The **Adam Optimizer** updates weights by minimizing the **Binary Cross-Entropy Loss**:

$$\boxed{Loss = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log(\hat{y}_i) + (1 - y_i)\log(1 - \hat{y}_i)]}$$

4. Algorithm Limitations

- **Black Box Nature:** It is difficult to explain exactly how an ANN arrives at a specific prediction compared to a Decision Tree.
- **Data Hungry:** Requires a large amount of labeled data to achieve high accuracy.
- **Computationally Intensive:** Training a deep ANN requires significant CPU/GPU power and time.

5. Methodology / Workflow

1. **Load Dataset:** Import the banking data and remove irrelevant identifiers (RowNumber, CustomerId, Surname).
2. **Data Preprocessing: * Encoding:** Use Label Encoding for Gender and One-Hot Encoding for Geography.
    - **Scaling:** Apply StandardScaler to normalize numerical features like Balance and Salary.
3. **Building the Architecture:** Create a Sequential model with two hidden layers using ReLU and an output layer using Sigmoid.
4. **Model Compilation:** Use the 'Adam' optimizer and 'binary_crossentropy' loss function.
5. **Training:** Fit the model on the training set for 50 epochs with a validation split.
6. **Performance Analysis:** Evaluate the model using a Confusion Matrix, Accuracy/Loss graphs, and an ROC Curve.

6. Performance Analysis

- **Accuracy/Loss Curves:** The model shows stable learning. A small gap between training and validation accuracy indicates a healthy generalization without significant overfitting.
- **Confusion Matrix:** Used to observe True Positives (correctly caught churners) and False Negatives (missed churners).
- **ROC-AUC Score:** With an area of approximately **0.86**, the model demonstrates strong discriminative power between staying and exiting customers.
- **Inference:** Customers with higher ages and lower active membership status show a higher probability of churning.

7. Hyperparameter Tuning

- **Epochs:** We increased epochs from 30 to 50 to allow the loss function to reach a more stable global minimum.
- **Batch Size:** A batch size of 32 was chosen to balance training speed and the stability of gradient updates.
- **Activation:** We chose ReLU for hidden layers to prevent the "Vanishing Gradient" problem and Sigmoid for the output to provide a probability-based classification.

Code:

1. Load and Explore Dataset

```
% Read the CSV into a table

data = readtable('Churn_Modelling.csv');


% Remove ID-based columns that don't help prediction

% RowNumber is col 1, CustomerId is col 2, Surname is col 3

data(:, 1:3) = [];


% Preview the first 5 rows in the Command Window

head(data, 5)
```

```
>> experiment_7
    CreditScore    Geography      Gender     Age    Tenure     Balance     NumOfProducts    HasCrCard    IsActiveMember    EstimatedSalary    Exited
    ___________    _________    __________    ___    ______    _________    _____________    _________    ______________    _______________    ______

        619        {'France'}    {'Female'}    42       2            0          1              1             1                1             1.0135e+05         1
        608        {'Spain' }    {'Female'}    41       1        83808          1              0             1                1             1.1254e+05         0
        502        {'France'}    {'Female'}    42       8    1.5966e+05         3              1             0                1             1.1393e+05         1
        699        {'France'}    {'Female'}    39       1            0          2              0             0                0                 93827         0
        850        {'Spain' }    {'Female'}    43       2    1.2551e+05         1              1             1                1                 79084         0
```

2. Encoding and Scaling

```
% Convert Geography and Gender to categorical labels

data.Geography = categorical(data.Geography);

data.Gender = categorical(data.Gender);


% Convert Categorical Geography to dummy variables (numeric matrix)

geog_dummies = dummyvar(data.Geography);


% Convert that matrix into a Table so we can join it

geog_table = array2table(geog_dummies, 'VariableNames', {'France', 'Germany',
'Spain'});
```

```
% Join: [First column, The new dummy tables, Everything from column 3 onwards]
% We skip column 2 because that was the old 'Geography' text column
data = [data(:,1) geog_table data(:, 3:end)];

% Gender also needs to be numeric for the ANN
% Since it's binary (Male/Female), we can just convert it to 1s and 2s
% or logicals. Let's make it numeric:
data.Gender = double(data.Gender);

% Separate Features (X) and Label (Y)
X = data{:, 1:end-1};  % Extracting numeric data from table
Y = categorical(data.Exited);

% Proceed with Splitting and Scaling (the rest of your code is fine!)
cv = cvpartition(size(X,1), 'Holdout', 0.2);
X_train = X(training(cv), :);
Y_train = Y(training(cv), :);
X_test = X(test(cv), :);
Y_test = Y(test(cv), :);

mu = mean(X_train);
sigma = std(X_train);
X_train = (X_train - mu) ./ sigma;
X_test = (X_test - mu) ./ sigma;
```

3. **Define Neural Network Architecture**

```
% Get the number of features (columns) in your training data
```

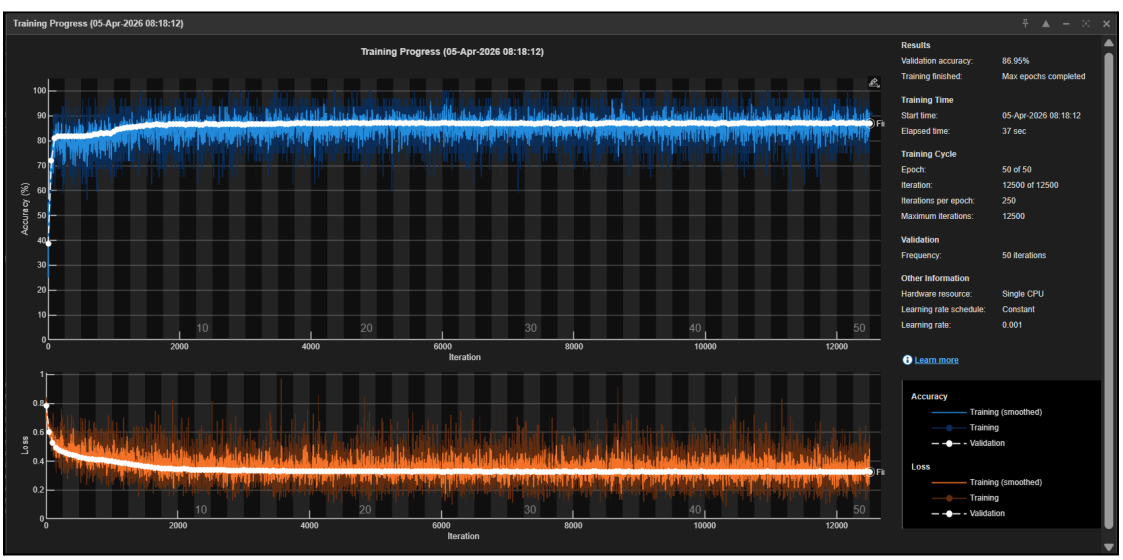
```
numFeatures = size(X_train, 2);
layers = [
    % Input Layer: Tells the network how many features to expect
    featureInputLayer(numFeatures, 'Name', 'Input')

    % First Hidden Layer: 6 neurons with ReLU activation
    fullyConnectedLayer(6, 'Name', 'Hidden1')
    reluLayer('Name', 'ReLU1')

    % Second Hidden Layer: 6 neurons with ReLU activation
    fullyConnectedLayer(6, 'Name', 'Hidden2')
    reluLayer('Name', 'ReLU2')
    % Output Layer: 2 neurons (for 0:Stayed and 1:Exited)
    fullyConnectedLayer(2, 'Name', 'Output')
    softmaxLayer('Name', 'Softmax')
    classificationLayer('Name', 'Classification')
];
```

```
X_test =

   -1.0445   -0.9954   -0.5783    1.7245   -1.0941   -1.4347    1.3823   -1.2288    0.8046    0.6457    0.9701   -1.4933
    0.8456    1.0044   -0.5783   -0.5798    0.9139    0.1983    1.0373   -1.2288    0.8046    0.6457    0.9701    1.2330
   -1.4473   -0.9954   -0.5783    1.7245   -1.0941   -0.0899   -0.3424   -1.2288   -0.9161    0.6457   -1.0307    0.3275
   -0.7036   -0.9954    1.7291   -0.5798    0.9139    0.1983    0.3474   -0.0998    0.8046   -1.5486    0.9701    1.3582
   -1.9120    1.0044   -0.5783   -0.5798   -1.0941    1.1589    1.0373    0.7374   -0.9161   -1.5486   -1.0307    1.4144
    1.8991    1.0044   -0.5783   -0.5798   -1.0941    0.9668   -1.0323    0.8798   -0.9161   -1.5486   -1.0307    1.6421
    1.3000   -0.9954    1.7291   -0.5798   -1.0941   -0.6662   -0.3424    0.5272    0.8046    0.6457    0.9701    0.4600
   -0.6727   -0.9954    1.7291   -0.5798    0.9139   -0.2820    0.0025    1.1150    0.8046   -1.5486   -1.0307   -0.2385
   -1.4369   -0.9954   -0.5783    1.7245   -1.0941    2.5998   -0.3424   -1.2288   -0.9161    0.6457   -1.0307   -1.7156
    0.9075   -0.9954    1.7291   -0.5798    0.9139    1.8313   -1.0323    0.9175    4.2461    0.6457   -1.0307   -1.2499
    1.3103    1.0044   -0.5783   -0.5798   -1.0941   -0.6662   -1.0323   -1.2288   -0.9161    0.6457   -1.0307    0.6332
   -1.4886    1.0044   -0.5783   -0.5798   -1.0941   -0.4741   -0.3424    0.2205   -0.9161    0.6457    0.9701    1.0300
   -0.0323   -0.9954   -0.5783    1.7245   -1.0941    0.4865    0.0025   -1.2288    2.5253    0.6457    0.9701    1.2908
   -1.3027    1.0044   -0.5783   -0.5798   -1.0941   -0.2820    1.7272   -1.2288    0.8046    0.6457   -1.0307    0.1655
    1.2690   -0.9954   -0.5783    1.7245    0.9139    0.1983    1.3823    0.4214   -0.9161   -1.5486    0.9701   -0.6188
   -0.7036   -0.9954    1.7291   -0.5798    0.9139   -0.6662   -1.3773    0.1985   -0.9161    0.6457    0.9701   -1.5691
   -1.7675   -0.9954    1.7291   -0.5798    0.9139   -0.3780    1.3823    0.2610   -0.9161    0.6457   -1.0307   -0.0116
    0.3602   -0.9954    1.7291   -0.5798    0.9139   -0.8583   -0.6874    0.2241   -0.9161   -1.5486    0.9701   -0.6452
   -1.1581   -0.9954    1.7291   -0.5798    0.9139    0.0062    0.6924    0.5053    0.8046    0.6457   -1.0307   -1.2698
    0.3498   -0.9954    1.7291   -0.5798   -1.0941    0.8707    1.7272    0.7994   -0.9161    0.6457    0.9701    1.7076
   -0.2595   -0.9954    1.7291   -0.5798    0.9139    1.6392   -1.7222    1.1544   -0.9161    0.6457   -1.0307   -0.9285
    1.8371    1.0044   -0.5783   -0.5798   -1.0941    0.1983    0.6924   -1.2288    0.8046    0.6457   -1.0307    1.2416
    1.4343   -0.9954    1.7291   -0.5798    0.9139    0.0062    0.6924    0.7745    0.8046    0.6457    0.9701    0.4233
   -0.6314   -0.9954    1.7291   -0.5798   -1.0941    1.0629    0.0025    1.0965    0.8046    0.6457    0.9701   -1.1355
    0.3292    1.0044   -0.5783   -0.5798   -1.0941    0.3904    0.0025    0.7909   -0.9161    0.6457    0.9701    1.6238
    0.4221    1.0044   -0.5783   -0.5798   -1.0941   -0.7623    0.0025   -0.5722   -0.9161    0.6457   -1.0307    0.4547
   -1.7055   -0.9954   -0.5783    1.7245   -1.0941   -1.7228    0.0025    0.5872   -0.9161    0.6457    0.9701   -0.8010
   -0.0013    1.0044   -0.5783   -0.5798    0.9139   -0.1859    0.0025    0.4878   -0.9161   -1.5486   -1.0307   -1.3175
   -1.1891    1.0044   -0.5783   -0.5798   -1.0941    0.8707    1.3823   -1.2288   -0.9161    0.6457   -1.0307    0.8672
    0.6803   -0.9954   -0.5783    1.7245    0.9139    0.2944    1.0373   -1.2288    0.8046    0.6457   -1.0307    1.4057
   -1.1478    1.0044   -0.5783   -0.5798    0.9139   -0.1859   -1.0323    0.8191   -0.9161    0.6457   -1.0307   -0.0256
   -0.1976   -0.9954    1.7291   -0.5798   -1.0941   -0.5701   -0.3424    0.7491   -0.9161   -1.5486   -1.0307    0.2190
```

4. Model Compilation, Training, and Performance Benchmarking

```
options = trainingOptions('adam', ...
    'MaxEpochs', 50, ...            % Equivalent to epochs=50
    'MiniBatchSize', 32, ...        % Equivalent to batch_size=32
    'InitialLearnRate', 0.001, ...  % Standard learning rate for Adam
    'Shuffle', 'every-epoch', ...   % Shuffles data to prevent ordering bias
    'ValidationData', {X_test, Y_test}, ...
    'Plots', 'training-progress', ... % THIS OPENS THE INTERACTIVE GRAPH
    'Verbose', false);
```

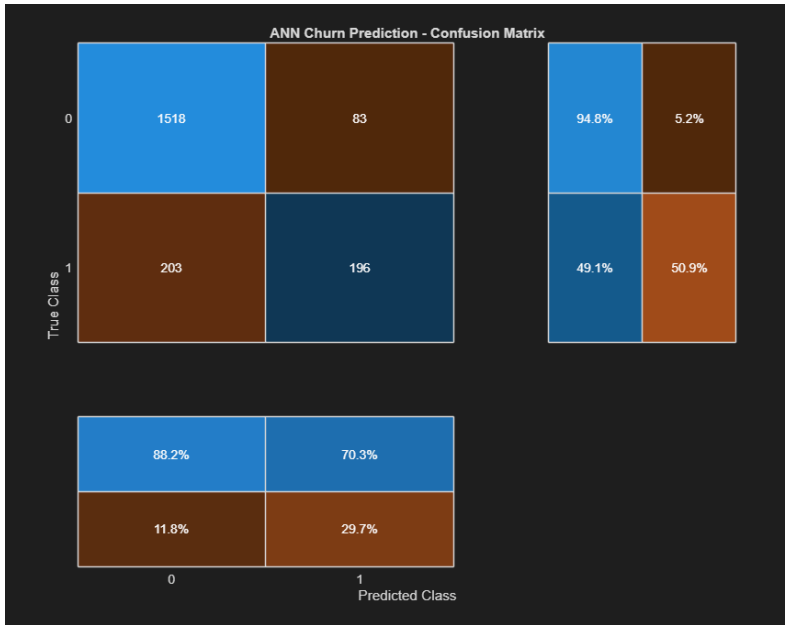
```
% Start Training - This will open a new window with real-time graphs
net = trainNetwork(X_train, Y_train, layers, options);
```



5. Create the Confusion Matrix

```
% 1. Predict Classes (for Confusion Matrix)
Y_pred = classify(net, X_test);
```

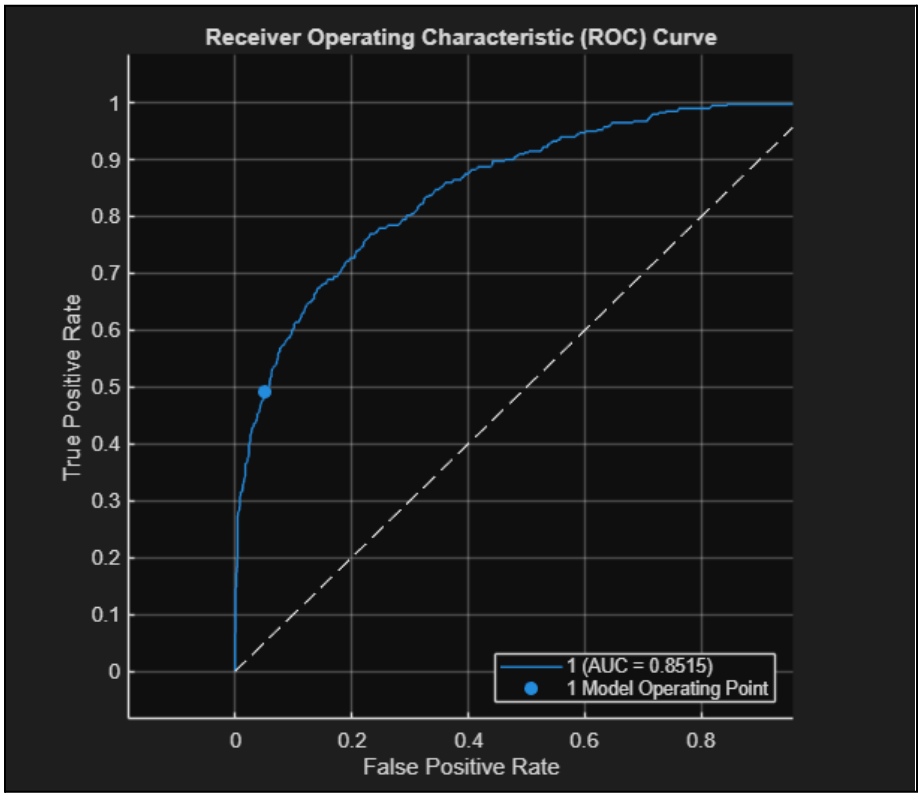
```
% 2. Create the Confusion Matrix

figure;

cm = confusionchart(Y_test, Y_pred);

cm.Title = 'ANN Churn Prediction - Confusion Matrix';

cm.RowSummary = 'row-normalized';

cm.ColumnSummary = 'column-normalized';
```



6. Model Validation: Confusion Matrix and Classification Metrics

```
% 3. Create the ROC Curve

% FIX: predict() returns only ONE output argument (the scores)

scores = predict(net, X_test);


% '1' represents the 'Exited' label in the Churn dataset

roc_obj = rocmetrics(Y_test, scores(:,2), '1');

figure;

plot(roc_obj);
```

```
title('Receiver Operating Characteristic (ROC) Curve');

grid on;


% Display Final Accuracy in Command Window

accuracy = sum(Y_pred == Y_test) / numel(Y_test);

fprintf('Final Test Accuracy: %.2f%%\n', accuracy * 100);
```



CONCLUSION

K-Means and Hierarchical Clustering were successfully implemented to segment mall customers. K-Means proved to be more efficient for rapid segmentation, while the Hierarchical Dendrogram provided a clear visual justification for the number of clusters chosen. The resulting segments allow for data-driven marketing strategies tailored to specific consumer groups.